

Experiment 6: Pattern Classification Using Perceptron Model

1. Aim

To implement the Perceptron Learning Algorithm for binary pattern classification using Python.

2. Software / Apparatus Required

- MATLAB
- MATLAB Editor
- MATLAB Command Window

3. Theory

The Perceptron is a single-layer neural network used for binary classification. It was introduced by Frank Rosenblatt.

The Perceptron calculates the net input as:

$$z = W^T X + b$$

where:

- **W** = weight vector
- **X** = input vector
- **b** = bias
- **z** = net input

A step activation function is used:

$$Y = +1, \text{ if } z \geq 0$$

$$Y = -1, \text{ if } z < 0$$

The predicted output is compared with the target output. If the prediction is incorrect, the weights are updated using the Perceptron learning rule:

$$W(\text{new}) = W(\text{old}) + \eta(T - Y)X$$

where:

- η = learning rate

- **T** = target output
- **Y** = predicted output

The training process is repeated until all patterns are correctly classified or the maximum number of epochs is reached.

The Perceptron is suitable for binary classification problems where the classes are linearly separable.

4. Algorithm

1. Start.
2. Initialize the weights and bias.
3. Define the training input patterns.
4. Assign target outputs to the input patterns.
5. Add the bias input to the training data.
6. Calculate the net input using the weights and input pattern.
7. Apply the step activation function.
8. Compare the predicted output with the target output.
9. Calculate the error.
10. If the classification is incorrect, update the weights.
11. Repeat the process for all training patterns.
12. Continue training until all patterns are correctly classified or the maximum number of epochs is reached.
13. Test the trained Perceptron.
14. Calculate the classification accuracy.
15. Display the final weights.
16. Plot the input patterns and decision boundary.
17. Stop.

This follows the algorithm provided in the experiment handout.

5. Learning Parameters

The following parameters are used:

- Learning rate = **0.1**
- Maximum epochs = **100**
- Class 1 target = **+1**
- Class 2 target = **-1**

The weights are initialized randomly, with `rng(1)` used so that the results are reproducible.

6. Decision Boundary

The decision boundary is obtained from:

$$W_0 + W_1X_1 + W_2X_2 = 0$$

Therefore:

$$X_2 = -(W_0 + W_1X_1) / W_2$$

The MATLAB program plots the two classes along with the learned decision boundary.

7. Expected Result

The Perceptron correctly classifies the training patterns and achieves high accuracy because the given patterns are linearly separable.

8. Applications

1. Email spam detection
2. Optical Character Recognition (OCR)
3. Medical diagnosis
4. Face recognition
5. Credit approval

9. Advantages

1. Simple and easy to implement.
2. Fast for simple classification problems.
3. Efficient for linearly separable datasets.
4. Requires simple mathematical computations.

The handout specifically identifies the Perceptron as simple, fast, easy to implement, and efficient for linearly separable datasets.

10. Limitations

1. It cannot solve the XOR problem.
2. It supports only binary classification in its basic form.
3. It requires linearly separable training data for convergence.

11. Conclusion

The Perceptron Learning Algorithm was successfully implemented in MATLAB for binary pattern classification. The Perceptron learned a decision boundary that correctly separated the two classes of linearly separable input patterns and achieved 100% classification accuracy.

The experiment demonstrates that a single-layer Perceptron can effectively perform binary classification when the input patterns are linearly separable.